Functions

~ G. Anushiya Rachel
Assistant Professor, CSE
Shiv Nadar University Chennai

Function Definition

- Header

$$def\ func_name(arg1, arg2, \ldots\ argN):$$

  - To define a function, start with the keyword def.
  - This is followed by the name of function and comma separated list of identifiers called **parameters** and terminated with a colon.
    - Values passed to the function – **arguments**
- Body
  - Contains the sequence of steps to be executed
- ***Note: A function must be defined before it is called but can be defined anywhere in the program.***

Function Definition and Call

- Example of function definition

```
>>> def mul(x, y):
        ans = x * y
        return ans
```

- Function call

```
>>> mul(2, 4)
8
>>> xy = mul(2, 4)
```

Function Call through Variable

- Function can be assigned to a variable and the variable can be used to call the function

```
def one():
        print('one')
def two():
        print('two')
def three():
        print('three')
```

```
a = input('Enter 1, 2 or 3: ')
if a == 1:
        call_func = one
elif a == 2:
        call_func = two
else:
        call_func = three
call_func()
```

Types of Functions

- Value-returning function – called for its return value
  - Multiple values returned in a tuple
- Non-value-returning function – called for its side effect

```
>>> def hello(name):
        print('Hello ', name)

>>> hello('John')
Hello John
```

Positional and Keyword Arguments

- Positional argument – When calling a function, an argument is assigned to a particular parameter based on its position in the argument list.

```
def mortgage_rate(amount, rate, term)

monthly_payment = mortgage_rate(350000, 0.06,  20)
```

- Keyword argument – When calling a function, the arguments are specified by parameter name

```
def mortgage_rate(amount, rate, term)

monthly_payment = mortgage_rate(rate=0.06, term=20, amount=350000)
```

Default Argument

- Default argument is one that can be optionally provided.

```
def mortgage_rate(amount, rate, term=20)
```

```
monthly_payment = mortgage_rate(35000, 0.62)
```

- The third argument, “term” is optional.
  - If omitted, the parameter “term” will take up the default value of 20.
  - If a third argument is provided, “term” will take up that value.

Variable Scope

- A **local variable** is a variable that is only accessible from within a given function. Such variables are said to have **local scope**.

- A **global variable** is a variable that is defined outside of any function definition. Such variables are said to have **global scope**.
  - The keyword ***global*** can be used to declare a global variable inside a function.

Variable Scope – Example

```
def func1():
    n = 10
    print('n in func1 = ', n)
```

```
def func2():
    n = 20
    print('n in func2 before call to func1 = ', n)
    func1()
    print('n in func2 after call to func1 = ', n)
```

```
>>> func2()
n in func2 before call to func1 =  20
n in func1 =  10
n in func2 after call to func1 =  20
```

Variable Scope – Example

```
def func1():
    # n = 10
    print('n in func1 = ', n)
```

```
def func2():
    n = 20
    print('n in func2 before call to func1 = ', n)
    func1()
    print('n in func2 after call to func1 = ', n)
```

```
>>> func2()
n in func2 before call to func1 =  20
Traceback (most recent call last):
  .
  .
    print('n in func1 = ', n)
NameError: global name 'n' is not defined
```

Variable Scope – Example

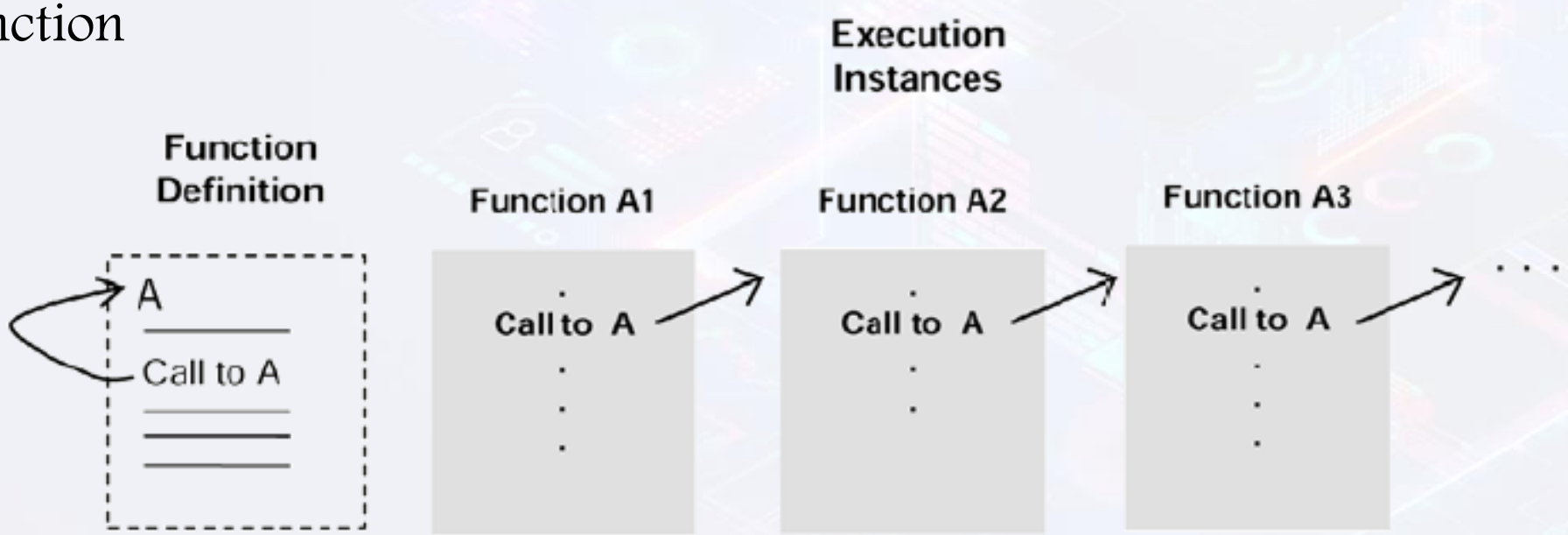
```
max = 100  # global variable

def func1(count):
  if count < max:
  .

def func2(count):
  for i in range(1, max):
  .
```

global variable `max` accessed

global variable `max` accessed

Recursion

- Recursive problem solving – A problem is repeatedly broken down into similar subproblems, until the subproblems can be directly solved without further breakdown.
- Recursive function – A function that calls itself
  - Function execution instance that calls another execution instance of the same function



Example – Factorial

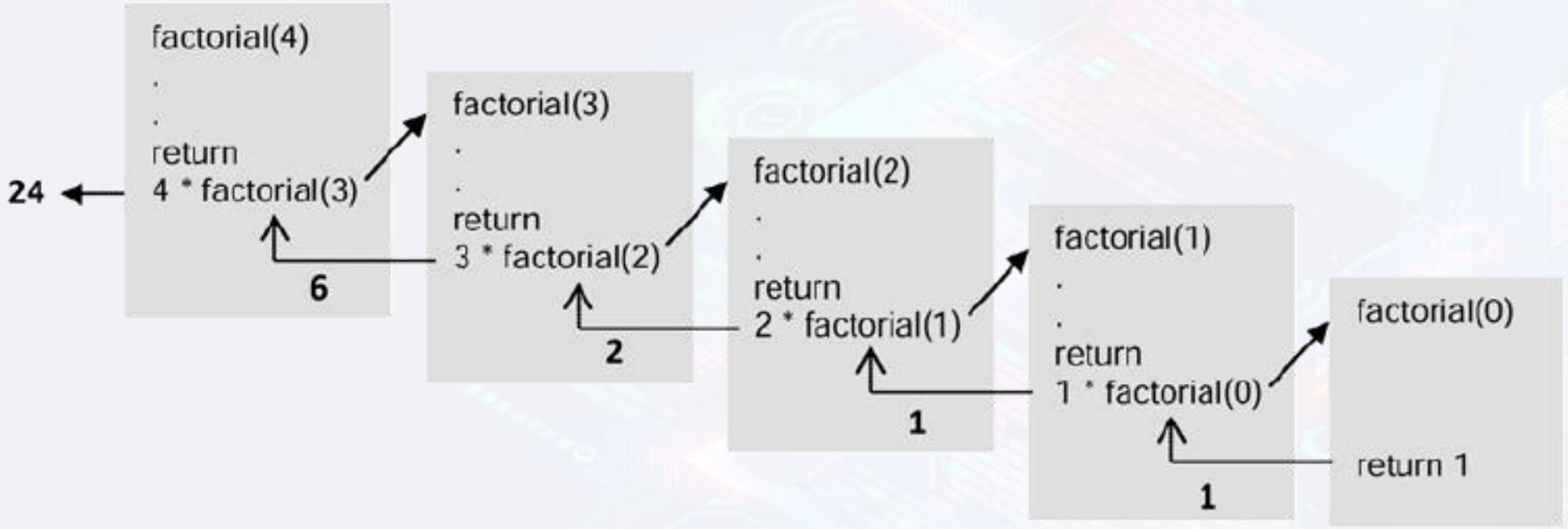
- $7! = 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$
- $6! = 6 \times 5 \times 4 \times 3 \times 2 \times 1$
- Therefore, $7! = 7 \times 6! = 7 \times 6 \times 5!$ etc…

$$factorial(n) = \begin{cases} 1, & n = 0 \\ n * factorial(n-1), & otherwise \end{cases}$$

- Recursive factorial function:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n*factorial(n-1)
```

Factorial Recursive Instance Calls



Programming Exercises

- Write Python programs for the following:
  1. Obtain an integer from a user and check if the number is a perfect number or a happy number or both. Write a function to check if the number is perfect and another to check if it is happy. (Perfect number – Number whose divisors, except itself, add up to itself. E.g. 28; Happy number – number for which the sum of the squares of the digits eventually equals 1. E.g. 203 is happy since $2^2 + 0^2 + 3^2 = 13; 1^2 + 3^2 = 10; 1^2 + 0^2 = 1$)
  2. Write a recursive function to count down from N.
  3. Write a recursive function to find the sum of digits of an integer.
  4. Write a recursive function to find the sum of the elements of a list.
  5. Write a recursive function to delete a digit in a specified position in a number.

Programming Exercises

6. Write a menu driven program to perform the following operations on a given sentence. Write a separate function for each of the following:
   - Reverse the sequence of words in the sentence.
   - List the characters in the string, without duplicates.
   - Find the word count and character count (including spaces).
   - Check if the string contains a given substring. If no substring is provided, check if the string contains "ing".
   - Encode the string by replacing each character in position i of the string by the character at position $i + 1$ and $i + 2$ alternately (if you reach the end of the string, go back to the beginning).
   - Exit

Programming Exercises

7. Write a function that adds 'ing' to the end of a given string, provided that the string has more than 3 characters. If the string already ends with 'ing', add 'ly' instead. If there are less than 3 characters, leave the string as it is.
8. Write a function that identifies if a given string is a valid IP address, using regular expressions. (An IP address is generally four numbers separated by dots. Each number can range from 0 to 255.)
9. Write a program to obtain a user's name, employee ID, PAN, and account number. Write functions to verify if each of these are valid using regular expressions. The conditions for validity are as follows:
    - Name: Only alphabets (upper or lower case), space, and dot
    - Employee ID: 5-digit number starting with 2, followed by any 4 digits
    - PAN: Contains 10 characters, where the first 5 are alphabets, followed by 4 digits, followed by 1 alphabet
    - Account number: A 10-digit number starting with 120